

Esercizio B1

Potrebbe sembrare uno dei classici esercizi basati su reti di flusso. In effetti, è possibile risolvere l'esercizio in questo modo, ma non è il metodo più efficiente. Il grafo risultante avrebbe $|V| = 2 + n + 6 = n + 8$ nodi e $|E| = n + 6 + 2n = 3n + 6$ archi. Una visita di tale grafo avrebbe costo $O(|V| + |E|)$, ovvero un costo pari a $O(n)$. Il flusso totale $|f^*|$ ha valore n . Quindi, secondo il limite di Ford e Fulkerson, il costo totale dell'algoritmo è $O(n^2)$.

È possibile tuttavia risolvere il problema con costo $O(n)$, utilizzando un approccio greedy.

Innanzitutto, si contano le richieste per ogni taglia, utilizzando un vettore di appoggio *richieste*[]. Questo passo potrebbe essere interpretato come un ordinamento dei partecipanti utilizzando Counting Sort; è più efficiente che ordinare i partecipanti tramite un algoritmo basato su confronti, che avrebbe costo $O(n \log n)$.

Per ogni taglia i , partendo dalla 1, si calcola quante magliette rimangono, sottraendo *richieste*[i] da $T[i]$. Se tale valore è negativo, le magliette non sono sufficienti per la particolare taglia, e bisogna utilizzare magliette della taglia successiva, sottraendo le magliette che mancano da $T[i + 1]$. Se $T[i + 1]$ diventa negativo, non ci sono magliette a sufficienza per soddisfare le persone con taglia i -esima, e si ritorna **false**. Altrimenti, si passa alla taglia successiva. Se tutte le taglie sono soddisfatte, si ritorna **true**.

Come sottoprodotto di questo approccio, il vettore T contiene il numero di magliette rimaste, per ogni taglia.

La correttezza dell'algoritmo può essere provata ragionando su ogni taglia. Le magliette di taglia 1 servono a persone di taglia 1, e quindi tanto vale assegnarle tutte a loro. Se non bastano, l'unico modo è utilizzare magliette di taglia 2. Se non bastano ancora, non c'è nulla da fare. Una volta risolte le persone di taglia 1, si fa lo stesso ragionamento sulla taglia 2, e così via.

L'algoritmo ha costo lineare $\Theta(n)$.

```
boolean isDoable(int[] T, int[] P, int n)
```

```
    int[] richieste = new int[1...5]
    for i = 1 to 5 do
        richieste[i] = 0
    for j = 1 to n do
        richieste[P[j]] = richieste[P[j]] + 1
    int i = 1
    boolean doable = true
    while doable and i ≤ 5 do
        T[i] = T[i] - richieste[i]
        if T[i] < 0 then
            T[i + 1] = T[i + 1] + T[i] % Poichè T[i] è negativo, questo corrisponde ad una sottrazione
            if T[i + 1] < 0 then
                doable = false
    return doable
```

Soluzione alternativa È possibile ordinare il vettore P , e poi assegnare una maglietta alla volta, sottraendo la maglietta $P[j]$ o $P[j] + 1$ a seconda della disponibilità. Il costo di questo algoritmo è $O(n \log n)$.

Esercizio B2

L'esercizio è identico all'esercizio 4 del 5/6/2014. La soluzione corrisponde alla funzione `partition( $H, n, k$ )` descritta in quel compito. Sarebbe stato completamente accettabile citare la soluzione e passare al prossimo esercizio.

Esercizio B3

Richiedendo di elencare tutte le possibili somme, è necessario utilizzare la tecnica di backtrack. Scriviamo una funzione wrapper che chiama la funzione `allSumsRec(int[] A, int[] S, int target, int i)`, dove S è uno stack contenente i valori che formano la somma, $target$ è il valore della somma da ottenere, i è l'indice dell'elemento di A che stiamo considerando.

Ad ogni chiamata, ci sono due possibilità: inseriamo $A[i]$ nello stack, lo sottriamo da $target$ senza modificare l'indice i perché il valore può essere riutilizzato; oppure non inseriamo $A[i]$ nello stack e passiamo al prossimo indice ($i - 1$). Se si raggiunge un target negativo ($target < 0$) oppure si finiscono i valori ($i = 0$), si compie un'operazione di backtrack. Se $target$ raggiunge il valore 0, si stampa il contenuto dello stack.

Il costo è pari a $O(v \cdot 2^{v+n})$, in quanto è necessario $O(v)$ per stampare ed ogni passo si provano due possibilità: o si diminuisce il valore o si diminuisce il numero di elementi. Ovviamente gran parte dell'albero viene potato, quindi questo è un limite superiore.

```
allSumsRec(int[] A, int[] S, int target, int i)
```

```
    if target == 0 then
        | print(S)
    else if target > 0 and i > 0 then
        | allSumsRec(A, S, target, i - 1)
        | S.push(A[i])
        | allSumsRec(A, S, target - A[i], i)
        | S.pop()
```

```
allSums(int[] A, int n, int v)
```

```
    STACK S = Stack()
    int[] S = new int[1...v]
    allSumsRec(A, S, v, n)
```
